

MCUBoot 開發者指南：從原始碼編譯到客製化安全啟動配置

原創

於 2026-02-05 00:33:47 發布

CC 4.0 BY-SA 版權

MCUBoot 開發者指南：從原始碼編譯到客製化安全啟動配置

【免費下載連結】[mcuboot](#)

32 位元微控制器的安全啟動！

專案地址：https://gitcode.com/gh_mirrors/mcub/mcuboot

MCUBoot 是一款專為 32 位元微控制器設計的安全啟動解決方案，它透過驗證韌體完整性和真實性來保護裝置免受惡意程式碼攻擊。本指南將帶你完成從原始碼編譯到客製化安全啟動設定的整個過程，幫助你快速部署安全可靠的啟動環境。

一、環境準備與原始碼獲取

1.1 安裝必要依賴

在開始之前，請確保系統已安裝以下工具：

CMake 3.13.1 或更高版本

Python 3.6 或更高版本

交叉編譯工具鏈（根據目標平台選擇）

Git 版本控制工具

1.2 取得 MCUBoot 源碼

使用下列命令克隆 MCUBoot 倉庫：

```
bash
```

```
1 | git clone https://gitcode.com/gh_mirrors/mcub/mcuboot
2 | cd mcuboot
```

二、編譯 MCUBoot 韌體

2.1 配置編譯選項

MCUBoot 支援多種建置系統，以 Zephyr 為例，進入 Zephyr 編譯目錄並配置目標板：

```
bash

1 | cd boot/zephyr
2 | west build -b <目標板型号> .
```

常用目標板設定檔位於 `boot/zephyr/boards/` 目錄下，例如 `nrf52840dk_nrf52840.conf`。

2.2 關鍵編譯參數說明

CMake 設定檔 `boot/zephyr/CMakeLists.txt` 中定義了核心編譯選項：

加密演算法選擇：支援 RSA、ECDSA、Ed25519 等多種簽章演算法

分區佈局：配置 `slot0`、`slot1` 和 `scratch` 分區大小

偵錯選項：啟用詳細日誌輸出（`CONFIG_MCUBOOT_LOG_LEVEL`）

優化等級：預設-O2 優化，可依需求調整

2.3 執行編譯

```
bash

west build -b nrf52840dk_nrf52840 .
```

編譯完成後，韌體檔案位於 `build/zephyr/zephyr.bin`。

三、安全啟動核心配置

3.1 理解鏡像結構

MCUBoot 使用特定的鏡像格式確保安全啟動，下圖展示了壓縮鏡像的結構組成：

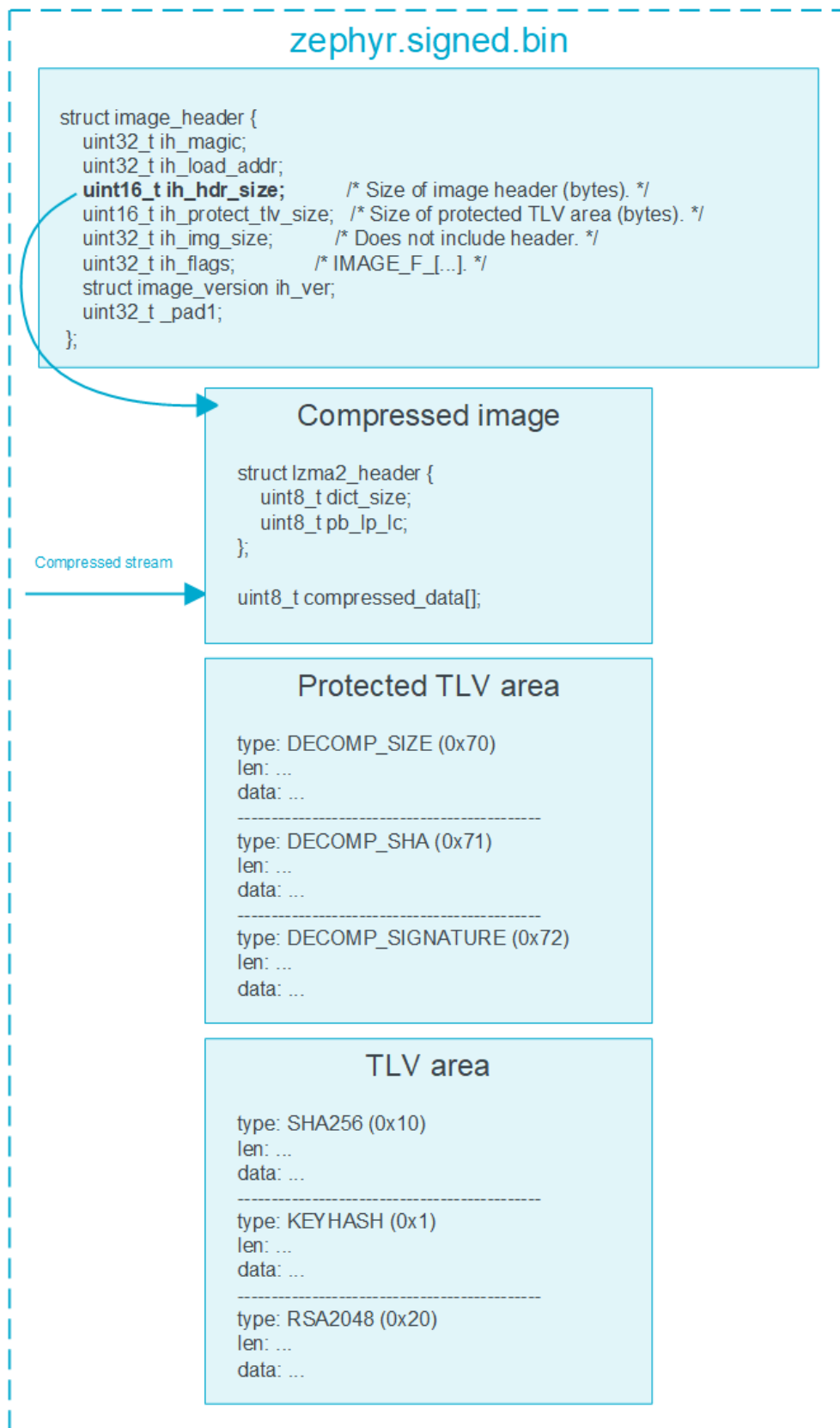


image 包含四個主要部分：

image header ：包含魔數、大小和版本訊息

壓縮資料 ：使用 **LZMA** 演算法壓縮的韌體內容

保護 TLV 區域 ：包含解密大小、哈希和簽名訊息

普通 TLV 區域 ：儲存額外元資料如 **SHA256** 雜湊和金鑰訊息

3.2 密鑰管理配置

3.2.1 產生簽章金鑰

使用內建工具產生 EC256 金鑰對：

預設金鑰檔案儲存在專案根目錄，如 root-ec-p256.pem 和 root-ec-p256-pkcs8.pem 。

3.2.2 配置加密金鑰

編輯 boot/bootutil/include/bootutil/enc_key_public.h 設定加密公鑰，支援 AES-KW、RSA 和 ECC 等加密方式。

3.3 分區佈局配置

修改設備樹檔定義 Flash 分割區：

dts

```
&flash0 {
    partitions {
        compatible = "fixed-partitions";
        #address-cells = <1>;
        #size-cells = <1>;

        boot_partition: partition@0 {
            label = "mcuboot";
            reg = <0x00000000 0x00010000>;
        };
        slot0_partition: partition@10000 {
            label = "image-0";
            reg = <0x00010000 0x00070000>;
        };
    };
};
```

四、高級客製與優化

4.1 多鏡像支援配置

修改 boot/bootutil/include/bootutil/bootutil.h 中的 BOOT_IMAGE_NUMBER 巨集定義，支援多重應用程式鏡像啟動：

c

```
#define BOOT_IMAGE_NUMBER 2 // 支持两个应用镜像
```

4.2 故障注入防護

啟用故障注入防護功能，編輯設定檔開啟：

c

```
#define MCUBOOT_FIH_PROFILE_HIGH 1
```

相關實作位於 `boot/bootutil/src/fault_injection_hardening.c` 。

4.3 運行時鉤子函數

透過實現鉤子函數自訂啟動行為，範例程式碼位於 `samples/runtime-source/zephyr/hooks/`，支援：

啟動前初始化

鏡像驗證回調

升級完成通知

五、測試與驗證

5.1 本地模擬測試

使用內建模擬器測試啟動流程：

bash

```
1 | cd sim
2 | cargo run --bin mcuboot-sim
```

5.2 硬體燒錄與調試

使用 `west` 工具燒錄韌體：

透過 J-Link 或 OpenOCD 調試，調試腳本位於 `scripts/jgdb.sh` 。

六、常見問題解決

6.1 編譯錯誤：缺少加密函式庫

確保 TinyCrypt 或 mbedTLS 庫已正確配置，檢查 `boot/zephyr/CMakeLists.txt` 中的加密庫路徑設定。

6.2 啟動失敗：簽章驗證錯誤

確認密鑰檔案路徑正確

檢查鏡像簽章是否使用正確金鑰

驗證 TLV 區域完整性

6.3 分區空間不足

調整分割區大小，確保每個 slot 至少包含：

應用韌體大小

TLV 區域（約 1KB）

狀態管理區域（根據扇區數計算）

七、參考資源

官方文件：[docs/index.md](#)

API 參考：[boot/bootutil/include/bootutil/bootutil.h](#)

設定範例：[boot/zephyr/prj.conf](#)

測試計畫：[docs/testplan-zephyr.md](#)

透過本指南，你已掌握 MCUBoot 的編譯、設定和自訂方法。合理使用這些功能可以建構安全、可靠的嵌入式設備啟動環境，有效防範韌體篡改和惡意攻擊。